TP2 - Modèle de langage neuronal

IAAA - ML

1 Objectif

L'objectif de ce projet est de programmer un modèle de langage neuronal à l'aide d'un perceptron multi-couches. Ce modèle de langage prend en entrée les plongements de k mots consécutifs et produit en sortie une distribution de probabilité sur l'ensemble du vocabulaire. Contrairement au projet précédent, nous ne ferons pas le calcul du gradient à la main, mais utiliserons la librairie `pytorch` et les outils de dérivation automatique qu'elle propose.

2 Le classifieur

Le cœur du modèle est constitué d'un perceptron multicouche C composé d'une couche d'entrée $\mathbf{x}$, d'une couche cachée $\mathbf{h}$ et d'une couche de sortie $\mathbf{y}$. Les paramètres du modèle sont regroupés dans deux matrices : $\mathbf{W}$ et $\mathbf{U}$ et deux vecteurs : $\mathbf{b}_1$ et $\mathbf{b}_2$. $\mathbf{W}$ et $\mathbf{b}_1$ permettent de calculer les valeurs de la couche cachée à partir de la couche d'entrée tandis que $\mathbf{U}$ et $\mathbf{b}_2$ permettent de calculer les valeurs de la couche de sortie à partir de la couche cachée.

Étant donné une séquence de mots $m_1 \dots m_n$, le classifieur C prend en entrée les plongements des k mots $\mathbf{m}_{i-k} \dots \mathbf{m}_{i-1}$ et produit en sortie une distribution de probabilité sur l'ensemble des mots du vocabulaire V.

On considère que les plongements de chaque mot sont de dimension d. La taille de la couche d'entrée, notée $\mathbf{x}$ vaut par conséquent $d_x = k \times d$ et la couche de sortie a pour dimension $|V|$, la taille du vocabulaire ! La dimension de la couche cachée est arbitraire, on la notera d_h. Les dimensions des deux matrices $\mathbf{W}$ et $\mathbf{U}$ sont donc $d_h \times d_h$ pour $\mathbf{W}$ et $d_h \times |V|$ pour $\mathbf{U}$.

Le calcul de la couche caché peut être décomposée en deux étapes :

— une étape linéaire : $\mathbf{h}' = \mathbf{x}\mathbf{W} + \mathbf{b}_1$

— suivie d'une étape non linéaire : $\mathbf{h} = \mathit{ReLU}(\mathbf{h}')$

De même, le calcul de la couche de sortie peut aussi être décomposée en deux étapes :

— une étape linéaire : $\mathbf{y}' = \mathbf{h}\mathbf{U} + \mathbf{b}_2$

— suivie d'une étape non linéaire : $\mathbf{y} = \mathit{Softmax}(\mathbf{y}')$

La fonction $\mathbf{y} = \mathit{softmax}(\mathbf{y}')$ permet de transformer un vecteur de valeurs réelles $\mathbf{y}'$ en un vecteur de probabilités $\mathbf{y}$. Chaque composante $\mathbf{y}_i$ de $\mathbf{y}$ est calculée de la manière suivante :

$$\mathbf{y}_i = \frac{\exp(\mathbf{y'}_i)}{\sum_{j=1}^{|V|} \exp(\mathbf{y'}_j)}$$

Autrement dit, la fonction $\exp(\cdot)$ rend la valeur de $\mathbf{y}'_i$ positive, et ensuite chaque valeur est normalisée par la somme de toutes les valeurs du vecteur $\mathbf{y}'_j$. Cela garantit que $\mathbf{y}_i$ est bien une probabilité car $\sum_{i=1}^{|V|} \mathbf{y}_i = 1$.

2.1 Fonction de perte

La fonction de perte utilisée pour mettre à jour les matrices de paramètres $\mathbf{W}$ et $\mathbf{U}$ est l'entropie croisée. Étant donné deux distributions de probabilité discrètes r et y, définies sur le même ensemble fondamental $\Omega = \{1, \ldots, N\}$, l'entropie croisée $H(r, y)$ se calcule de la manière suivante :

$$H(r, y) = -\sum_{i=1}^{N} P_r(i) \log_2(P_y(i))$$

Si on considère que la distribution r est la distribution de référence (la distribution que l'on aimerait que notre modèle calcule) et que y est la distribution calculée par le modèle, $H(r, y)$ mesure d'une certaine façon l'erreur que l'on commet en utilisant la distribution y à la place de la distribution r.

Dans notre cas r est une distribution bien particulière, il s'agit de la représentation *one-hot* d'un mot du vocabulaire. Etant donné un vocabulaire V, où chaque mot correspond à un indice compris entre 1 et $|V|$, la représentation one-hot du mot d'indice i est un vecteur de dimension $|V|$ dont toutes les composantes sont à 0 sauf la composante i qui vaut 1. Un tel vecteur binaire peut être vu comme la représentation (pas très économique) du i-ème mot du vocabulaire. Il peut aussi être vu comme une distribution de probabilité où toute la masse de probabilité est concentrée sur un seul mot : celui d'indice i.

Lorsque r est une représentation one-hot, tous les termes s'annulent sauf un. Dans ce cas, l'entropie croisée se calcule de manière très économique, elle vaut :

$$H(r, c) = -\log_2(\mathbf{y}_i)$$

où i est l'indice du mot attendu (le mot correspondant à la valeur 1 dans la représentation one-hot r) et est $\mathbf{y}_i$ la i-ème composante de la couche de sortie du classifieur (la probabilité qu'associe le modèle au mot que l'on souhaite prédire via la fonction softmax).

2.2 Données

Pour réaliser vos expériences, vous utiliserez les données suivantes :
— `train_12k.bpe` un corpus d'apprentissage constitué de 4 683 231 tokens.
— `test_12k.bpe` un corpus de test constitué de 584 689 tokens.
— `val_12k.bpe` un corpus de test constitué de 580 004 tokens.

— un fichier d'embedding qui associe un embedding à chacun des 12k tokens. Plusieurs tailles d'embeddings sont disponibles. Les noms des fichiers sont de la forme `bpe_embeddings_12k_n.txt` où `n` représente la taille des embeddings.

Afin de ne pas passer trop de temps à écrire le code pour la lecture du fichier de plongements, nous fournissons aussi une classe `Vocab.py`. Cette classe prend le nom du fichier de plongements en paramètre de son constructeur et charge tous les plongements dans une matrice `pytorch`. Des fonctions permettent de convertir facilement un mot au format textuel vers son indice, son plongement, ou sa représentation one-hot.

3 Développement du modèle

1. Ecrire la fonction de préparation des données qui prend en entrée un fichier de texte T ainsi qu'un dictionnaire associant à tout mot du vocabulaire, un indice [1]. Cette fonction produit en sortie une liste dont les éléments sont des listes de $k+1$ mots consécutifs de T représentés par leur indices.

   Par exemple, si le corpus se présente de la façon suivante `a b c d e f` et que le mot `a` correspond à l'indice 1, le mot `b` correspond à l'indice `2` ...et que k vaut 3, les listes `X` et `Y` se présenteront sous la forme suivante :

   ```
   X = [[1,2,3,4],[2,3,4,5],[3,4,5,6]]
   ```

2. Ecrire la boucle d'apprentissage qui parcours un certain nombre de fois la liste des exemples regroupés en batch et, pour chaque batch, construit le graphe de calcul de la fonction de perte, calcule son gradient et effectue un pas d'optimisation. Il est intéressant, pour chaque itération (epoch) d'afficher la valeur moyenne de la fonction de perte sur les données d'apprentissage et de développement. La perte sur les données d'apprentissage doit décroître si l'apprentissage se passe bien. L'évolution de la perte sur les données de développement va permettre d'arrêter le processus d'apprentissage grâce au *early stopping*. N'oubliez pas de mélanger les batches aléatoirement entre chaque itération.

3. Sauvegarder les paramètres dont les valeurs ont été optimisées lors de l'apprentissage. Vous utiliserez pour cela la fonction `torch.save`.

4 Evaluation

L'évaluation du modèle se fera à l'aide de deux métriques, la perplexité et l'accuracy sur un texte à trou. Vous programmerez aussi une bouche de génération afin de vous faire une idée du type de texte que votre modèle peut générer.

1. Vous pourrez utiliser la classe `Vocab` qui permet de lire un fichier de plongements et d'accéder à l'indice d'un mot ainsi qu'à son plongement.

1. Ecrire le programme `perplexite.py` qui prend en entrée un modèle M (le fichier sauvegardé à l'issue de l'apprentissage) et un texte T et calcule la perplexité de T étant donné les probabilités données par M.
2. Ecrire le programme `remplis_trous.py` qui prend en entrée un texte à trou et un fichier de tokens candidat. Le texte à trou se présente sous la forme d'une séquence de tokens précédé d'un entier. L'entier indique la position du trou. Par exemple :

   ```
   16 <s> <s> <s> par ce rude hiver , les vieux chênes des ard #en
   #nes brûlaient à grande flamme </s> </s> </s>}
   ```

   Dans cet exemple, il faut retrouver le token 16 (`brûlaient`). Vous trouverez trois exemples de textes à trous dans les données fournies (les fichiers `file_1.mask`, `file_2.mask` et `file_3.mask`)

   La recherche du token est coûteuse car il faut remplacer le token éliminé par tous les tokens du vocabulaire. Afin d'accélerer le processus, nous utiliserons un sous ensemble du vocabulaire. Pour cela, nous vous fournissons dans le répertoire `tokens_candidats_textes_a_trou_12k` dix fichiers comportant chacun cent tokens. Avant de lancer la recherche du token maximisant la probabilité, vous devez vérifier que le token correct est dans la liste, sinon le rajouter et en elever un de sorte à avoir toujours 100 tokens.

   Pour évaluer votre modèle sur les textes à trous, vous utiliserez la métrique du rang normalisé vue dans le cours (Données pour les TP).
3. Ecrire le programme `genere.py` qui prend en entrée un modèle M et génère du texte par tirage aléatoire dans les distributions de probabilités calculées par le classifieur.

4.1 Résultats attendus

Les résultats que vous obtiendrez dépendent de la valeur des hyper-paramètres que vous utiliserez. Ces derniers étant nombreux, vous ne pourrez pas recourir à du *grid search* afin de trouver leurs valeurs optimales. Nous vous fournissons ci-dessous un jeu de valeur pour les hyper-paramètres et les résultats que vous devriez obtenir.

Hyper-paramètre	Valeur
nombre de couches cachées	1
taille de la couche cachée	256
taux d'apprentissage	10^{-3}
taille des batches	256
taille de l'historique	3
taille des embeddings	32
optimiseur	Adam
fonction non linéaire de la première couche	ReLU
initialisation des embeddings	aléatoire
mise à jour des embeddings	non

En partant de ces hyper-paramètres, vous devriez obtenir une perplexité de l'ordre de 5. Il ne s'agit pas d'un résultat absolu car il dépend de l'initialisation aléatoire des embeddings et de la permutation aléatoire des batches à chaque itération. En ce qui concerne les textes à trous, vous devriez atteindre un rang normalisé de l'ordre de 0.98.

5 Quelques pistes à creuser

Le modèle de langage décrit ci-dessus utilise des plongements qui ont été appris à l'avance. Les pistes ci-dessous visent à explorer d'autres manières d'utiliser des plongements. Trois pistes sont proposées, parmi lesquelles vous en choisirez deux. La première est obligatoire (les deux autres en dépendent). Vous devez donc choisir (1,2) ou bien (1,3).

1. Il est possible de faire en sorte que le modèle de langage apprenne ses propres plongements. Pour cela, les plongements doivent devenir des paramètres du modèle, qui seront mis à jour lors de l'apprentissage. Pour cela, il faut enrichir les paramètres du modèle d'une matrice $\mathbf{E}$ apprenable de dimension $V \times d$ où V est la taille du lexique et d la dimension des plongements. La création du vecteur d'entrée consiste à concaténer les lignes de $\mathbf{E}$ correspondant aux indices des k mots du prefixe (on pourra utiliser la fonction `torch.concat`).
2. Nous avons maintenant deux manière de construire des plongements, soit à l'aide de l'algorithm SGNS vu dans le projet précédent, soit à l'aide du modèle de langage programmé dans ce projet. L'idée est de comparer ces plongements. Vous pourrez commencer par évaluer leurs performances respective sur la tâche de calcul de similarité du projet précédent. Vous analyserez ensuite la répartition des mots dans les deux espaces d'embeddings : est ce que les mots les plus proche d'un mot cible t dans le premier espace sont aussi les mots les plus proches de t dans le second ? Vous pourrez aussi utiliser la projection t-sne pour visualiser en deux dimensions les deux espaces.
3. Lorsque le modèle de langage apprend ses propres plongements (la matrice $\mathbf{E}$), on peut partir d'une matrice aléatoire, comme on fait pour les autres matrices de paramètres, mais on peut aussi partir de de plongements pré-entrainés, par exemple par l'algorithme SGNS. L'objectif de cette piste est de comparer ces deux façon de faire par rapport à la performance des modèles de langage (est ce que les perplexités des modèles de langage sont différentes), la qualité des plongements (est ce que les plongements appris avec initialisation obtiennent de meilleurs résultats sur la tâche de similarité) et à la vitesse d'apprentissage (est ce que l'apprentissage est plus rapide si on part de plongements pré-entrainés ?).